The University Of Azad Jammu and Kashmir, Muzaffarabad
Department of Software Engineering

Student Name	**Asim Shahzaman**
Roll Number	**2025-SE-26**
Program	**BS Software Engineering**
Course Title	**Software Engineering Fundamentals**
Course Code	**SEC – 532**
Course Instructor	**Engr. Ghulam Rabbani Butt**
Semester	**3rd**
Session	**2025–29**
Assignment No	**01**
Assignment Title	**University course registration system**
Assigned On	**Oct 20, 2026**
Due Date	**Oct 28, 2026**

1. University course registration system

This report applies the concepts from Lecture 02, “From Real World Concept to Software Object,” to a University Course Registration System. The purpose of the system is to help students register for course sections offered during a semester. It checks prerequisites and seat availability, records enrollment, and supports updating a student’s transcript after final grades are available.

In object-oriented design, a software object is a computational model of a real-world or invented entity. A useful object combines state (data or attributes), behavior (methods or operations), and identity (what makes one instance distinct from another). A class defines the shared structure and behavior of objects, while an object is a particular instance of a class.

2. Candidate Conceptual Classes

The following candidate classes were identified by examining the nouns and noun phrases in the registration scenario and checking them against conceptual class categories such as roles of people, transactions, events, rules, and records.

Candidate Class
1. Student
2. Course
3. CourseSection
4. Semester
5. Enrollment
6. Prerequisite
7. Transcript
8. Instructor
9. Classroom
10. Schedule

3. Attributes and Methods of Three Classes

3.1 Student

Attributes (state):

- studentID : String
- name : String
- email : String
- registeredSections : List
- cumulativeGPA : Decimal

Methods (behavior):

- register(section : CourseSection) — requests registration in a course section.
- drop(section : CourseSection) — withdraws from a registered section.
- hasCompleted(course : Course) : Boolean — checks whether a course has been completed.

- getTranscript() : Transcript — retrieves the student's transcript.
- updateGPA() — recalculates the cumulative GPA when appropriate.

3.2 CourseSection

Attributes (state):

- sectionID : String
- course : Course
- semester : Semester
- instructor : Instructor
- capacity : Integer
- currentEnrollment : Integer
- schedule : Schedule
- room : Classroom

Methods (behavior):

- isFull() : Boolean — determines whether the section has reached its capacity.
- addStudent(student) — adds a student when registration rules permit.
- removeStudent(student) — removes a student from the section.
- hasSeatAvailable() : Boolean — checks whether a seat remains.
- getRemainingSeats() : Integer — returns the number of unoccupied seats.

3.3 Enrollment

Attributes (state):

- enrollmentID : String
- student : Student
- section : CourseSection
- enrollmentDate : Date
- status : String (e.g., active, dropped, completed)
- finalGrade : String (optional until grading is complete)

Methods (behavior):

- create() — creates an enrollment record after registration checks succeed.
- drop() — marks the enrollment as dropped when a student withdraws.
- complete(finalGrade) — records completion and the final grade.
- isActive() : Boolean — indicates whether the enrollment is active.

4. Abstraction Decisions

Abstraction means keeping only the information relevant to the problem and deliberately leaving out unnecessary details. For the Student class, this report includes the student ID, name, email, registered sections, and GPA because these details support registration and academic records. Details such as favorite color, shoe size, music taste, and sleeping pattern are excluded because they do not help the registration process. The CourseSection class includes its course, semester, instructor, capacity, schedule, and room because these are needed to determine whether a student can join a section. A grade is treated as an Enrollment attribute in this simple model; a larger system could model grades separately if more rules or history were required. These choices keep the model purposeful rather than attempting to represent every detail of university life.

5. Invented Software Object

A useful invented software object is RegistrationController. It does not represent a physical object in the university; instead, it coordinates the registration use case. When a student requests a course section, RegistrationController checks prerequisites, asks CourseSection whether seats are available, creates an Enrollment record if the checks pass, and coordinates updates to the relevant objects. This separates the workflow from domain data and avoids placing every responsibility into one “System” class.

6. Conclusion

First, I studied the university course registration system and identified the main concepts involved in the registration process. Then, I identified ten candidate conceptual classes, including Student, Course, CourseSection, Semester, Enrollment, Prerequisite, Transcript, Instructor, Classroom, and Schedule. I briefly explained the purpose of each class. After that, I selected three important classes—Student, CourseSection, and Enrollment—and described their attributes and methods. Next, I explained the abstraction decisions by identifying the relevant information required by the system and excluding unnecessary details. Finally, I identified RegistrationController as an invented software object and explained how it coordinates the registration process by checking prerequisites, verifying seat availability, and creating enrollment records. Through this assignment, I learned how real-world concepts can be represented as software objects and how classes, attributes, methods, and abstraction help in designing an effective software system.